

レポート課題 3

05-191009 奥村真司

問 1

動作例

```
$ ocamlc -c strSet.mli
$ ocamlc -c strSet.ml
$ ocamlc -c sort.ml
$ ls -F *.cm*
sort.cmi  sort.cmo  strSet.cmi  strSet.cmo
$ ocamlc -o sort strSet.cmo sort.cmo
$ ls -F sort
sort*
$ ./sort << END
> bbb
> ccc
> aaa
> bbb
> END
aaa
bbb
bbb
ccc
-----

.cmo をインタプリタで利用

$ ocaml
      OCaml version 4.05.0

# #load "strSet.cmo";;
# StrSet.empty ;;
- : StrSet.t = <abstr>
# StrSet.count_sub ;;
Error: Unbound value StrSet.count_sub
# open StrSet;;
# add "abc" empty;;
- : StrSet.t = <abstr>
-----

.mli をコンパイルしない

$ ocamlc -c strSet.ml
File "strSet.ml", line 1:
Error: Could not find the .cmi file for interface strSet.mli.
```

```

-----

change file order (at link time)

$ ocamlc -c strSet.mli
$ ocamlc -c strSet.ml
$ ocamlc -c sort.ml
$ ocamlc -o sort sort.cmo strSet.cmo
File "_none_", line 1:
Error: Required module 'StrSet' is unavailable

-----

use OCamlMakefile

$ ls -F *t.m*
sort.ml  strSet.ml  strSet.mli
$ make
make[1]: ディレクトリ '/home/tansei/UTokyo/Funclogic/week3' に入ります
ocamlc \
    \
        -o sort \
            strSet.cmo sort.cmo
make[1]: ディレクトリ '/home/tansei/UTokyo/Funclogic/week3' から出ます
$ ls -F sort
sort*

```

考察

上記の動作例の通り試してみた。スライド内で注意があったように、strSet.mli をコンパイルしないと cmi が生成されないせいで strSet.ml のコンパイルがうまくいかなかった。また、リンク時に sort.cmo を strSet.cmo より前にもってくると sort.cmo 内で strSet が用いられているのでうまくコンパイルできなかった。OCamlMakefile は [github\(https://github.com/mmottl/ocaml-makefile/blob/master/OCamlMakefile\)](https://github.com/mmottl/ocaml-makefile/blob/master/OCamlMakefile) から落としてきた。Makefile 内のような短い記述で実行ファイルが生成できて便利だった。

問 2

動作例

```

# Stack.empty;;
- : 'a Stack.t = <abstr>
# Stack.push 1 (Stack.empty);;
- : int Stack.t = <abstr>
# Stack.push 2 (Stack.push 1 (Stack.empty));;
- : int Stack.t = <abstr>
# Stack.size (Stack.push 2 (Stack.push 1 (Stack.empty)));;
- : int = 2
# Stack.pop (Stack.push 2 (Stack.push 1 (Stack.empty)));;
- : int * int Stack.t = (2, <abstr>)

```

考察

シングニチャ STACK は'a t と抽象化し、スタックは一方からの出し入れができればよいので Stack の具体的な内部実装としては list を用いた。

問 3

動作例

```
# module IntMultiset = Multiset2 (OrderedInt);;
module IntMultiset :
  sig
    type t = Multiset2(OrderedInt).t
    val empty : t
    val add : OrderedInt.t -> t -> t
    val remove : OrderedInt.t -> t -> t
    val count : OrderedInt.t -> t -> int
  end
# let e = IntMultiset.empty;;
val e : IntMultiset.t = <abstr>
# let s = IntMultiset.add 5 (IntMultiset.add 5 (IntMultiset.add 2 e));;
val s : IntMultiset.t = <abstr>
# IntMultiset.count 5 s;;
- : int = 2
# IntMultiset.count 2 s;;
- : int = 1
# IntMultiset.count 5 (IntMultiset.remove 5 s);;
- : int = 1
```

考察

二分木で実装した。あるノード v に対してそのノードの左の部分木に含まれるノード w の値は v の値よりも小さく右の部分木は v の値以上であるように設計した。このように設計したので count ではノードの値が探索している値より大きいと左の部分木のみ再帰的に探索するようになっている。remove ではまず remove する対象のノードを探索し、見つかったらそのノードの右の部分木の最小値のノードをそこに挿入している。ここで注意すべきは左の部分木の最大値と入れ替えると、count が正しく動作しなくなるということである。例えば 4,3,3 の順で挿入したあとに 4 を削除すると、左の子の最大値と入れ替えた場合 3 の左の子が 3 になる。これは左の部分木内の頂点の値はすべて自分より小さいという条件に反し、count は両方の部分木について再帰的に計算する必要がでてくる。(これは FL 実験の目的とは離れた考察かもしれませんがバグった原因として面白かったので記述しました。)

問 4

動作例

```
# module MapIntInt = Map (Map_node (OrderedInt) (ValueInt));;
```

```

module MapIntInt :
  sig
    type t = Map(Map_node(OrderedInt)(ValueInt)).t
    val empty : t
    val add :
      Map_node(OrderedInt)(ValueInt).key_t ->
      Map_node(OrderedInt)(ValueInt).val_t -> t -> t
    val remove : Map_node(OrderedInt)(ValueInt).key_t -> t -> t
    val lookup :
      Map_node(OrderedInt)(ValueInt).key_t ->
      t -> Map_node(OrderedInt)(ValueInt).val_t
  end
  # open MapIntInt;;
  # let e = empty;;
  val e : MapIntInt.t = <abstr>
  # let e = (add 1 6 e);;
  val e : MapIntInt.t = <abstr>
  # let e = (add 2 15 e);;
  val e : MapIntInt.t = <abstr>
  # let e = (add 2 15 e);;
  val e : MapIntInt.t = <abstr>
  # let e = (add 3 19 e);;
  val e : MapIntInt.t = <abstr>
  # lookup 1 e;;
  - : Map_node(OrderedInt)(ValueInt).val_t = 6
  # lookup 3 e;;
  - : Map_node(OrderedInt)(ValueInt).val_t = 19
  # let e = (remove 1 e);;
  val e : MapIntInt.t = <abstr>
  # lookup 1 e;;
  Exception: Map(T).LookupError.

```

考察

問3で実装した二分木をベースに拡張した。二分木の各ノードはkeyとvalueのデータを保持している。keyのタイプはORDERED_TYPE,valueのタイプをVALUE_TYPEとした。次に二分木のノードのシグネチャMAP_NODE_TYPEとkeyのタイプとvalueのタイプを受け取って二分木のノードのモジュールを返すファンクタMap_nodeを定義した。二分木のノードのモジュールには、ノードからkeyとvalueを取り出す関数と、keyとvalueからノードを生成する関数と、ノードを2つもらってそれらをkeyで比較する関数を用意した。続いてkeyのタイプとvalueのタイプを受け取って二分木を返すファンクタのシグネチャMAPとMAPを満たすファンクタMapを定義した。Mapによってできる連想配列モジュールはempty,add,remove,lookupを備えている。Mapの内部では問3と同様の方針で二分木を実装している。

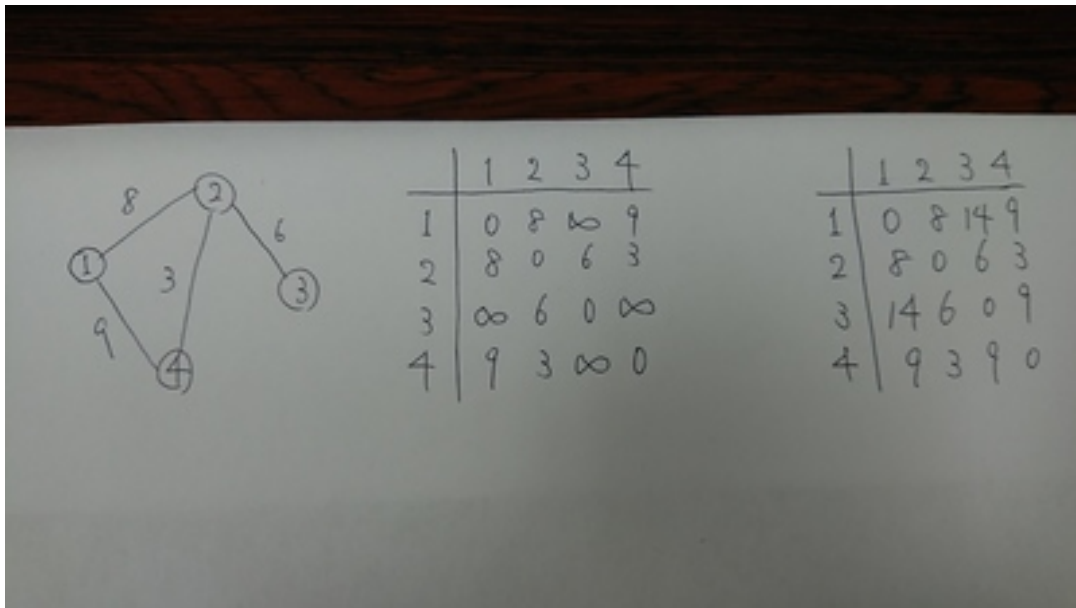
問 5

動作例

```
# module DistMatrix = Matrix(Distring);;
module DistMatrix :
  sig
    type t = Matrix(Distring).t
    val init : Distring.t list list -> t
    val add : t -> t -> t
    val mul : t -> t -> t
    val pow : t -> int -> t
    val output : t -> Distring.t list list
  end
# open DistMatrix;;
# let m = init [[Z 0;Z 8;INF;Z 9];
               [Z 8;Z 0;Z 6;Z 3];
               [INF;Z 6;Z 0;INF];
               [Z 9;Z 3;INF;Z 0]];;
val m : DistMatrix.t = <abstr>
# output (pow m 5);;
- : Distring.t list list =
[[Z 0; Z 8; Z 14; Z 9];
 [Z 8; Z 0; Z 6; Z 3];
 [Z 14; Z 6; Z 0; Z 9];
 [Z 9; Z 3; Z 9; Z 0]]
```

考察

行列とベクトルの演算を定義するのが課題要件であるが、ベクトルは列数 1 の行列とみなせるので行列と行列の演算を定義した。まず半環を受け取ってその行列モジュールを返すファンクタのシグネチャ `MATRIX` を定義した。初期化用のインターフェースとして半環を要素とするの `list` の `list` をもらって半環の行列を生成する `init`, そして課題要件の `add,mul` を定義し、出力用のインターフェースとして行列の内部表現を `list` の `list` に変換する `output` を定義した。`add,mul` 時の行列の型チェック用に `row,column` 関数を用意し、`add,mul` は内部に補助関数を大量に用意して再帰的に計算している。`add` ではまず行列の型チェックをし、1 行目とそれ以外に分割して再帰的に求めている。`mul` ではベクトルの内積の計算に帰着させるために `transpose` 関数で掛ける方の行列を置換してから計算している。`problem5.ml` 内で課題要件の 2 つの半環上の行列モジュールが定義されている。`DistMatrix` は半環 $(\mathbb{Z} \cup \infty, \infty, 0, \min, +)$ 上の行列モジュールで、動作例では下の画像のようなグラフでの各頂点間の最短距離を計算しており、正しく動作していることがわかる。各半環モジュールに `SEMIRING` シグネチャを指定すると半環の型が隠匿されて初期化が面倒になるのであえて指定はしていない。



発展 1

動作例

```

# let c1 = EConstInt (Eq.refl, 1);;
val c1 : int expr = EConstInt (<abstr>, 1)
# let c2 = EConstInt (Eq.refl, 2);;
val c2 : int expr = EConstInt (<abstr>, 2)
# let add12 = EAdd (Eq.refl, c1, c2);;
val add12 : int expr =
  EAdd (<abstr>, EConstInt (<abstr>, 1), EConstInt (<abstr>, 2))
# let ct = EConstBool (Eq.refl, true);;
val ct : bool expr = EConstBool (<abstr>, true)
# let cif = EIf (ct, add12, c2);;
val cif : int expr =
  EIf (EConstBool (<abstr>, true),
    EAdd (<abstr>, EConstInt (<abstr>, 1), EConstInt (<abstr>, 2)),
    EConstInt (<abstr>, 2))
# let cadd = EAdd (Eq.refl, c1, ct);;
Error: This expression has type bool expr
      but an expression was expected of type int expr
      Type bool is not compatible with type int
# eval add12;;
- : int value = VInt (<abstr>, 3)

```

考察

まず `apply` の型の `('a, 'b) equal -> 'a -> 'b` を実現するのにもっとも素朴な `('a, 'b) equal` の型としては `'a -> 'b` が考えられる。これで実装しようとする `refl` と `trans` は実装できるが `symm` が難しい (逆関数っぽいものをつくるのが難しい)。なのではじめてから双方向のデータをもたせておけばよいのでは? という発想から

`{right: 'a -> 'b; left: 'b -> 'a}` としてみるとすんなり実装できた。eval も、前回の課題の実装を少し変えただけでできた。Eq の証明がついているおかげで、式定義の時点でエラーになってくれるので例外を吐く処理の必要がなくなった。その代わりパターンマッチに漏れがあるとコンパイラが warning が出すが無視した。